What Breaks in Long-Lived Agents: An Eight-Month Operating Record of a Ten-Agent Fleet

Ravshan Nuraliev

Independent Researcher

October 2026
Preprint, not peer-reviewed.
DOI: 10.5281/zenodo.23404010

Abstract

Most agent benchmarks score a task in a fresh context, and newer memory benchmarks probe recall across sessions. Some costly failures can occur in one session but become visible and accumulate only over months. We report eight months of operating a fleet that grew to ten long-lived coding agents (nine on Claude Code) on four machines. First, *false zeros*: empty tool results taken as proof of absence. We list eighteen carriers and quantified cases such as 64 of 1,501 memory files silently dropped at a permission boundary. Second, verification needs a different position or instrument: a guard change that passed its author's check let through 52 commands its predecessor blocked. Third, we measure where a 1M-token window goes. The model's own output is the largest share (39%), and tool-server responses are below 1%. After a client update, idle compaction began to fire, visible only in its timing. A pre-registered blind study shows a full handoff brief restores 70.7% of probed facts after rotation, against 0% without it, but loses to the native summary under the same character limit. Fourth, we report negative results, including a mood classifier that did not beat "always neutral".

1 Introduction and Testbed

1.1 Why a production fleet

Most agent benchmarks score a task in a fresh context: one session, one goal, a score at the end [6]. Memory benchmarks go further. LongMemEval [12] and LoCoMo [7] test whether an assistant recalls facts across many sessions. They do it on curated conversations with a fixed answer key, and they measure the model's recall, not the infrastructure around it. The failures we spent most of 2026 on sit in that infrastructure. They appear in agents that live for months on real machines, carry memory between sessions, are rotated and compacted many times, and run scheduled jobs no one watches:

- an empty tool result read as proof that something does not exist (§2);
- a verifier that shares the horizon and the instrument of the thing it verifies, and therefore agrees with it (§3);

- a context window filled by things nobody budgeted for, and a session that keeps running on an old prompt after the prompt file changed on disk (§3, §4);
- a handoff that captures the right facts and fails to deliver them (§4);
- self-approved prompt rules that kept growing until an audit cut them back (§5).

Work on agent memory [8] and on agents that reflect on their own failures [10] addresses parts of this problem, but it is evaluated on benchmark episodes, not on months of operation of the same agents.

Each of these failures can occur in a single session, and none of them is a recall failure of the model. What months of operation added was repetition: we saw the same failure return across sessions and agents, and saw its consequences settle into memory, where later sessions retrieved them as facts. Our contribution is therefore narrow: an operating record of one fleet, a classification of the failures we found, and the measurement practice that found them. We report what we measured, what we changed, and what did not work. We do not claim a new memory method; where we tested our own handoff method against the client's built-in summary, it lost (§4).

1.2 The testbed

The fleet ran from February to October 2026, about eight months of continuous operation.

Agents and machines. Ten long-lived agents with their own persona and memory run on four machines:

- seven agents in containers on one cloud host (Node-Cloud-1…7);
- one agent on an x86 laptop (Node-Mac-x86) and one on an ARM laptop (Node-Mac-ARM), with a real browser and a residential network;
- one coding agent on a separate server (Node-Codex).

Figure 1 shows the layout.

Since 14 October the cloud host also runs a service agent with no persona (Node-Service-8). Non-conversational model calls from scheduled scripts on the cloud host, such as nightly reviews and correction analysis, are routed to it instead of to the agents' live sessions.

Isolation. Every cloud agent is the unmodified Claude Code CLI running in a Linux container, with the CLI version pinned per node. All eight agent containers on the cloud host have a read-only root filesystem; only a small set of home, state, log and temporary directories is writable. Five other containers (credential broker, approval kernel, host daemon, secrets broker, a local search engine) run separately with writable roots. A read-only root is not full isolation: each agent container also has writable bind mounts into directories shared with the host, which is how agents exchange files.

Automation. The cloud host runs 194 `systemd` user timers: nightly consolidation, memory maintenance, backups, health checks, report jobs. Node-Mac-x86 adds 82 loaded `launchd` jobs, schedules and daemons together.

Memory. Each agent writes observations to its own memory store, a relational database with a vector index for semantic search. On 15 October 2026 the cloud host held about 240,000 observation rows across 13 stores: one per cloud agent, one each for the two laptop nodes and the coding node, and three service stores for corrections, rules and lessons. Three of the cloud stores share a cloned early history of 60,672 identical rows; without the duplicates the host holds about 120,000 distinct

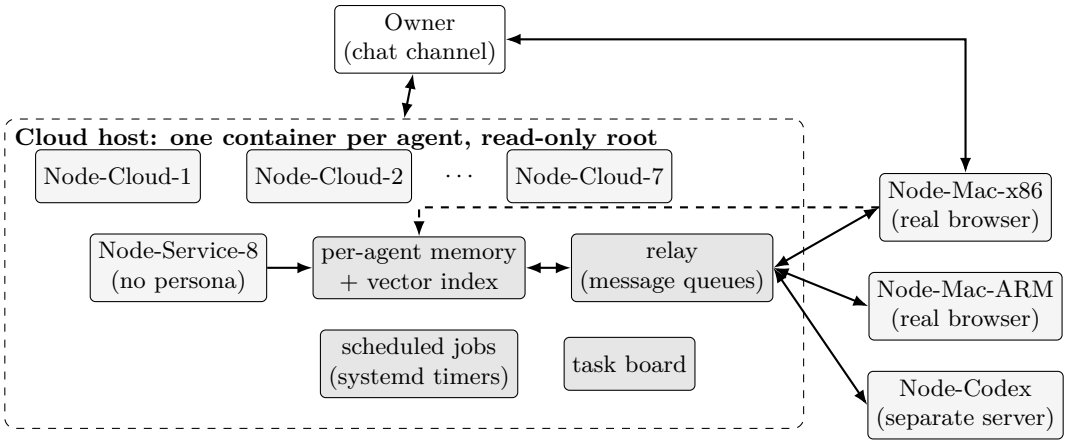


Figure 1: The fleet. Seven agents and a service node run in containers on one cloud host; two laptop agents with real browsers and one coding agent run outside it. Agents exchange work through a relay of message queues and keep their own memory stores. A check made from inside a container and a check made from the host see different things (§3). The dashed arrow is memory synchronisation from Node-Mac-x86.

rows. The vector index held about 521,000 vectors. Node-Mac-x86 keeps a further 40,502 rows in its own store. We do not add it to the host's total: until 15 October the two stores shared 14,986 rows.

Coordination. Agents exchange messages through a file-based relay. Long-running joint work lives in shared thread files, so that a message carries a path and not the content. One human operator sets direction, approves irreversible or paid actions, and owns all accounts.

1.3 How the fleet changed over eight months

What we mean by a long-lived agent. In this paper a long-lived agent is a named role whose identity files and memory store persist for months, while any single session of the client lasts from hours to a few days. A session ends when the operator layer rotates it with a handoff brief, or when the client compacts it. On Node-Mac-x86, 46 main sessions between mid-September and mid-October lasted a median of 10.5 hours of wall-clock time and at most 74 hours. Across the cloud agents, the operator layer recorded 20 rotations in about a week. No session lived for months; the agents did.

The configuration changed throughout. The October setup described in this paper is the last state of a system that changed throughout the eight months. The main steps are listed in Table 1:

Table 1: Main changes to the fleet, February to October 2026.

When (2026)	Change
February	First assistant sessions; the per-agent memory store starts on 8 February

When (2026)	Change
late April – early May	Five agents on one laptop, each with its own memory; first agent-to-agent channel; vector index
18 May – 11 June	Agents move into containers on a cloud host; seven cloud agents by 11 June
19–21 June	Two-way relay between the cloud host and the laptop node; handoff brief written at compaction on all seven; first version of the claim-structured brief
7 July	Verify-Before-Deny gate in log-only mode; automatic rotation before the native limit (then at 850K tokens)
21 July	Coding agent on a separate server
3 August	1M-token window in effect on all seven cloud agents
21–28 August	Board hygiene; the four-condition test rule (§3.4); second laptop node
September	An audit finds six cloud containers running non-Anthropic models through a router; such sessions continue in parts of the fleet until about 22 September
19–23 September	Small decision model in shadow mode (§5); rotation threshold lowered to 650K; Claude Opus 5.5; the build pipeline of §6 becomes the default
7 October	Learned prompt rules cut back to 20 per agent
10–14 October	Client 2.1.295 on four cloud agents, 2.1.284 on three; service node; idle compaction switched off

Which model and client each study used. Because models and client versions changed over the period, each result is tied to the configuration it was measured on. Study C (4 October) used Claude Opus 5.5 sub-agents as blind answerers, run from client 2.1.284, with a mechanical scorer. Its rotation events all came from Claude sessions, but 19 of its 29 compaction events came from sessions running two other model families, which probably wrote those native summaries (§4). The window budget (14 October) sampled sessions that ran mostly on Opus 5.5, on client versions 2.1.259 to 2.1.296. The command-guard and prompt-snapshot cases (14 October) ran on client 2.1.284, 2.1.295 and 2.1.296; the guard itself is a deterministic hook. For the Verify-Before-Deny replay (7 July) neither the model nor the client version of the replayed transcripts was recorded. The mood classifier (19 September) is the decision model `jev-1.13`. Knowledge tracing and the prompt-rule audit involve no model in the measured step.

1.4 Method and data policy

Every number in this paper is tied to a primary file: a transcript, a log, a pre-registration commit or a measurement report. The tie is recorded in a provenance table kept with the paper. The node

that extracts a number does not verify it. A second node re-checks it against the primary file with a different instrument, a rule we justify in §3. For Study C (§4) we also prepared a reproducibility package that recomputes every published number from per-unit counts.

We report measurements from October 2026 where they exist. Where we use an older measurement, we give its date and sample size. Small-sample results from early summer are not used. Node names are neutral; no client data, credentials, addresses or personal information appear in the paper.

1.5 Contributions

1. A field catalogue of *false zeros*, empty tool results taken as proof of absence. It includes eighteen carriers recorded in one night, quantified cases, a stated scope for an honest zero, and a deterministic Verify-Before-Deny gate with an account of what it did not measure (§2).
2. *Two horizons and two instruments*: cases in which a checker inside the same environment as the code it checked agreed with the defect, and a four-condition rule for smoke tests (§3).
3. A measured window budget for long sessions, the discovery of an idle-compaction mechanism in the client, and a pre-registered blind study of handoff briefs after rotation and after compaction. The study includes a rejected hypothesis (§4).
4. A hybrid decision architecture, in which code owns control, a calibrated decision model judges, and the LLM plans. The section also covers negative results, including self-approved prompt rules that grew to 50–61 per agent and had to be cut back to 20 (§5).
5. An observed operating record of the build pipeline and the task board, with the limits of records the agents write themselves (§6).

1.6 What the evidence supports

Table 2 states, for each main result, what our data support and what they leave open. The rest of the paper should be read against it: where a section seems to say more than the middle column, the table is the claim we stand behind.

Table 2: What each main result supports and what it leaves open.

Result	What the data support	What remains open
False zeros (§2)	The mechanisms and cases we observed, with their sizes in each case	How often false zeros occur among all tool calls, and how many we missed
Honest zeros and countermeasures (§2.4)	That the search wrapper separates an engine failure from an engine answer; the countermeasures we used in operation	Whether a search behind an honest zero was complete; whether the countermeasures beat leaving the check to the agent (not compared)
Verify-Before-Deny gate (§2.5)	How the rule behaves on 6,310 replayed production turns	Its precision in production, and whether it reduces false denials
Second position or instrument (§3)	Specific defects that a first check passed and a second position or instrument found	How often this happens, and the average effect on reliability

Result	What the data support	What remains open
Window budget (§4)	An estimate of where context growth went in 30 sampled sessions, reproduced by two parsers	The accuracy of single shares (the split is by characters, and thinking is not logged as text); transfer to other configurations
Idle compaction (§4)	Early compactions with a fixed idle signature on one client version; the mechanism in the client binary	Why it fired on one version and not the other; the live server settings
Study C (§4)	Recovery of probed facts under the tested conditions: the full brief after rotation, and the client's native summary against our brief under the same character limit	Whether recovered facts make the agent's later work better; the brief against a summary at an equal token budget
Small decision model (§5)	Its latency, price and accuracy on the tasks and labels we report, including where it lost to a trivial baseline	Transfer to other models and tasks
Operating record (§6)	Throughput and timing as recorded by the agents themselves	Value to the owner, time saved, cost per task

2 False Zeros: Field Cases and the Verify-Before-Deny Gate

2.1 Definition

A *false zero* is an empty or zero-valued tool result that an agent takes as evidence that an object, state or event does not exist, when the emptiness was produced by the measurement path and not by the world. The path can be a permission boundary, a locale, a glob, a filter, an encoding or a pipeline.

An *honest zero* is narrower than "not a false zero". It is an empty result from a path that was shown, in the same check, to be able to return a non-empty answer, and it holds only for the scope that path covers. A positive control shows that the path works. It does not show that the search was complete: the control file can be readable while the wanted file sits under other permissions, in another directory, or behind a filter the control did not pass through. A claim of an honest zero therefore has four parts: where the search looked, with which filters, which checks passed, and what remains unknown. Outside that scope the honest answer is "not found here", not "does not exist".

The two look identical at the output. `0`, `[]` and an empty string carry no information about which path produced them. An agent reading the output therefore has no signal to separate them. Its next step is usually a denial ("the file does not exist", "the service is not running") or an action built on the denial: rebuilding a file that is present, re-sending a message that was delivered, reporting a feature as missing. In a long-lived fleet these denials persist in memory and are retrieved later as facts.

We treat the false zero as a class of error, not as a list of bugs. The rest of this section gives the field evidence for treating it as one class, the cases we could quantify, the countermeasures we used, and a deterministic gate that targets the downstream symptom.

2.2 Eighteen carriers in one night

During one paired review night (5–6 September 2026) the two operator nodes logged eighteen distinct mechanisms that produced the same symptom:

1. a wrong path on disk;
2. a silenced stderr;
3. reading a slice of a file (tail instead of head);
4. a nested JSON key read at the wrong level;
5. a conclusion from a single sample;
6. a placeholder directory mistaken for an access denial (ACL);
7. file mtime taken as the record timestamp;
8. checking the source instead of the built artifact;
9. a counter used without reading the matched line (wrong in both directions);
10. a failure written to a log nobody reads;
11. a gap between two review buckets;
12. dating a breakage by the first error instead of the last success;
13. a host path used from inside a container;
14. `sudo` combined with home-directory expansion, which resolves to the superuser's home;
15. one's own instrument applied to another node's horizon;
16. a search run in the wrong directory;
17. a check that merged "no such file" and "not found";
18. a correct answer masked by a fallback (`|| echo`).

The list is reported as recorded that night. It is a catalogue of observed carriers, not an exhaustive or orthogonal taxonomy: items 1, 13 and 16 overlap, and item 9 can produce false positives as well as false zeros.

2.3 Quantified cases

The cases below were found at other times and have their own measurements. We report them separately from the list in §2.2.

Container–host permission boundary. Memory files written from inside a container receive the container's uid. In September 2026, 64 of the fleet's 1,501 memory files were unreadable by the host process that builds the recall index. The indexer caught the `PermissionError` and skipped the file. The files were never deleted; they silently dropped out of retrieval, with no log line.

Locale and non-Latin text. Under a POSIX locale, character counts and character classes over Cyrillic text were wrong by a factor of 1.7 to 40 (for example, `wc -m` reported 55,164 characters for a file with 32,367; a Cyrillic capital-letter character class matched 40 lines in a file with one capital letter). Literal string matches were unaffected, which is why the error passed unnoticed for weeks.

Hidden names. Backup files on the shared volume start with a dot. A default glob listed 13 files where `ls -a` listed 135, so 90.4% of the artifacts were invisible to the default pattern. This matters most during a rollback, when an agent lists backups before deciding whether one exists.

Symlinked queues. Two queue paths pointed to one directory. A naive walk counted 157 dead letters; the true number after deduplication by `realpath` was 108.

Encoding at a search boundary. The fleet's shared memory-search service decoded raw Cyrillic query strings as Latin-1. Four Cyrillic terms returned 0, 0, 0 and 0 results; the same terms URL-encoded returned 21, 40, 3 and 39; a Latin control term returned 37. The zeros were indistinguishable from honest empty results.

Two filters in a vector store. The memory reader applied two conditions to every vector query: an owner condition on a metadata field, and a 90-day recency window. A later writer stored the owner only in the record key and not in that field. New records failed the owner condition, and the older records that had the field were aging out of the window. For one node, 14,834 of 43,331 observations written between late July and mid-October 2026 were invisible to semantic search, while still present in the relational store.

Silent success in scheduled automation. An October audit of the cloud host's nightly code (20 small-model workers over 177 scripts and eight days of timer journals) produced 62 findings: 31 confirmed by a live check (one of them later refuted), 3 confirmed in code but not yet triggered, 28 plausible. A recurring pattern among the confirmed findings was a job that reported success while doing nothing: a memory sync that returned "nothing new" on every one of 1,276 runs and exited successfully; a morning briefing that failed for 29 days with exit code 0; a weekly job killed by a termination signal while its timer still showed success; an association step that created zero links in 330 of 363 runs.

2.4 Countermeasures that held

None of the fixes below is new on its own. In our operation they held when they were written into the tool protocol and applied every time; when they were left to the agent's judgement, the same failures came back. We did not compare the two regimes in a controlled way, so this is an operating observation, not a measured effect.

1. **Positive and negative control in the same call.** A check for an object includes a probe of a known-present object through the same path. If the control also returns zero, the path is broken, not the world.
2. **Visible stderr in counting pipelines.** A count produced by `ls | wc -l`, `find | wc -l` or `grep -c` is re-checked with stderr shown. "No such file" and "permission denied" are visible only there.
3. **Every empty result prints the path it searched.** This is the one countermeasure that is preventive rather than corrective.
4. **Relaxed retry.** The fleet's memory search returns only records that match every query word, so a zero from a multi-term query is retried with one or two terms before it is accepted. After the encoding incident above, the fleet also adopted the broader rule that a zero on its own proves nothing.
5. **Honest and suspicious zeros are separated in the tool protocol.** The fleet's web-search wrapper returns either `[ok] HONEST ZERO` or `[!] FALSE-ZERO WARNING` on an empty result, so the distinction reaches the model as text and not as an absence. The honest label is narrower than its name. It means that every search engine answered and none reported an error. It does not show the filters of the call (a minimum relevance score, a result cap, a language, a time range), and the wrapper applies its score filter after the engines answer, so a zero produced by the caller's own filter is also labelled honest. An engine that returns an empty page without reporting an error is not detected either. We therefore read the label as "the engines were reachable for this query", not as "nothing exists".

2.5 The Verify-Before-Deny gate

The countermeasures act on the measurement. The Verify-Before-Deny (VBD) gate acts on the symptom: a denial emitted without a check.

Mechanism. VBD is a deterministic Stop hook with no model in the loop. After a turn, it scans the final message for negative claims in two classes. EXISTENCE claims ("does not exist", "not found", "there is no such…") are high-signal and are checked with subject matching. STATUS claims ("broken", "not running", "hung") are noisy, because an agent honestly describes a problem it is actively fixing. A STATUS claim fires only when the turn made no tool call at all and the message carries no calibration marker. A firing re-injects one instruction to verify; the hook never fires twice on the same stop.

Offline evaluation. We replayed 6,310 real turns from 14 production transcripts (July 2026) through five successive versions (Table 3):

Table 3: Verify-Before-Deny gate, offline replay of 6,310 production turns: firings per version.

version	change	firings	rate
v0.1	explicit denial phrases only	31	0.49%
v0.2	subject matching for all denials	91	1.44%
v0.3	negation guard, calibration-marker exemption, inline-JSON strip	58	0.92%
v0.4	EXISTENCE / STATUS split	35	0.55%
v0.5	message-level calibration exemption	20	0.32%

These are firing rates, not verified false-positive rates. Version 0.2 is a regression: subject matching tripled the firings, and all 91 were STATUS denials made during active debugging. Splitting the two classes (v0.4) and exempting calibrated messages (v0.5) brought the rate below the first version. The 20 firings that remain in v0.5 match the target pattern: denials made with no tool call and no calibration marker. The node that ran the replay labelled them as the target class; no independent review checked whether each denial was actually false.

What we did not measure. The gate was deployed in log-only mode on three nodes. Its production precision is unknown, and whether it reduces false denials in production is not established. The only gate log we found holds two firings from one node, on 25 July and 19 August, which is too few to estimate anything. In October we found that its log had not been written: the log directory was not writable for the gate's uid, and the write error was swallowed. The tool built to catch false zeros produced one. We report this as a result: an observability layer needs the same positive control as the measurements it guards. Here that would be a canary write and a read-back at startup.

3 Positional Epistemology: Two Horizons and Two Instruments

3.1 The problem: a judge that shares the defect

A common way to check an agent's work is to ask another model to judge it [13]. In a single environment this tends to become an echo chamber. The judge reads the same files through the same tools, with the same permissions and the same blind spots. When the defect sits in the measurement path, as with the false zeros of Section 2, the judge reproduces it and confirms the wrong answer.

The fleet did not start from a theory. Over three months the operator nodes wrote down a number of separate rules, each after a concrete failure. Read together, they share one idea: a claim is checked from a different *position*, or with a different *instrument*, from the one that produced it. We present this as our own synthesis. The fleet's records keep these rules apart and never name them as one canon.

3.2 Two horizons

The fleet runs agents inside containers on a cloud host, and two operator nodes run on physical machines outside it. During a paired review night (5–6 September 2026) the two sides found that they could see different things. The cloud node sees its own assembled prompt from inside the container. The physical node sees the host: service units, the process table, other containers and their resource groups. The records describe the split of work as "by horizon, not by task".

The split paid off immediately. Working from inside, the cloud node located all 12 of 12 prompt layers that the outside view had reported missing. In the same night each side caught the other on a false zero: the cloud node caught the physical node four times, the physical node caught the cloud node twice, and three further corrections were about wording. The night's taxonomy of false zeros (Section 2) lists the pattern as two of its eighteen carriers: "own instrument against another's horizon" and "a host path read from inside a container".

3.3 Two instruments

The second half of the idea is to change the instrument, not only the reader.

Compiling is not importing. On 6 September a fix to the handoff retry called `time.sleep()` in a file that did not import `time`. A byte-compile check passed, and the error surfaced only when the module was imported as a whole. The rule that followed is to import the whole module after adding any new call, because a byte-compile does not resolve names.

A ladder of evidence for runtime configuration. The pair "process arguments versus transcript" in early drafts is too simple. The records describe a ladder with four rungs: a value in a configuration file is weaker than a launch argument, which is weaker than a literal in the shipped binary, which is weaker than the live consumption recorded in the session transcript. A process listing shows only the executable name. The rule was settled on 3 August 2026, when the top rung, a live transcript reading, showed one session at 347,177 input tokens.

3.4 When is a test evidence? Four conditions

A test that only ever passes proves nothing about the fix it is meant to protect. On 23 August 2026 two nodes wrote down, and each checked on its own work, the conditions under which a test counts as evidence for a fix. A test counts only if it

1. goes red on the pre-fix code,
2. is wired into the smoke suite,
3. sits, with the suite, at a permanent address, and
4. keeps the evidence it goes red on inside its own inventory.

Conditions 3 and 4 each came from a failure. A patcher kept in a temporary directory lived for four hours. Evidence kept in another component's directory disappeared after one clean-up, and the test's failure then looked like an ordinary skip.

3.5 Three cases from September and October 2026

A test bench that modelled the code. On the night of 19–20 September the two physical nodes and the cloud node checked a fix to a plugin key routine. The first bench on one node rebuilt the key itself instead of calling the function, and it reported 8/8 for a file whose real key fails one of the checked properties. A corrected bench that called the real function gave the same file 5/8. The cloud node's first test set failed in the same way: it guessed the digit order from the source text. The rule recorded that night is that a bench which *models* the logic it checks tests itself, not the code. It was caught twice in one night, by both sides independently.

A guard regression found by a second table. On 14 October a change (labelled F22) to the command guard, the hook that blocks dangerous shell commands, passed its author's own check: "everything the old guard blocked is still blocked". That check rested on the author's 69 commands. A second node built an independent table of 387 commands (352 generated cases and 35 everyday commands). On it, 52 commands that should be blocked were blocked by the previous guard and let through by the F22 version. These are regressions introduced by F22. They are not all of the guard's gaps: in total the F22 version let 72 dangerous commands through, and the previous guard 25. A follow-up fix restored the block on 51 of the 52, and a further rule closed the last one, a heredoc that appends to a shell start-up file. On the third node the live guard missed 4 of 11 dangerous probes that the pre-F22 guard had blocked. The fix was then checked on three separate sets, none of which found the regressions: a comparison bench of 358 dangerous-command cases (352 generated cases and 6 probes), where it showed zero regressions against the pre-F22 guard; 328 real working commands; and 53 benign commands added by the reviewer. The fix blocked none of the 328 and none of the 53.

A prompt snapshot that outlived its source. The fleet's command-line client can store the system prompt with the session and reuse it on resume. On 14 October we tested whether a changed prompt reaches a resumed session. Each prompt carried a distinct control word. A new session given ALPHA-7 answered correctly. The same session, resumed with a prompt carrying BRAVO-9 and the default snapshot setting, still answered ALPHA-7. A control run, resumed with the snapshot switched off and given CHARLIE-3, answered CHARLIE-3. A second run, using `--continue` instead of resume and two client versions, reproduced the stale answer. The test passed the prompt as a launch argument, not as a file. That the same mechanism ignores a prompt file

regenerated on disk at each start is our inference for the fleet's start-up path, supported by the client's own help text, not a separate measurement. The run used a throw-away session; the live agents were checked read-only and none was affected that day. A fix was installed on all affected start-up files the same night, with a checker that verifies it.

3.6 What carries over

Three practices generalise beyond this fleet. First, assign verification by position: whoever can see the host checks claims about the host, and whoever can see the assembled prompt checks claims about the prompt. Second, when a check passes, change the instrument before trusting it: import, do not only compile; read consumption, do not only read configuration. Third, admit a test as evidence only under the four conditions of Section 3.4. In each of the three cases, the first check passed and was wrong, and a second position or a second instrument found the defect.

4 The Window Budget, Idle Compaction, and a Blind Handoff Benchmark

4.1 Where the context window goes

Setting. Every agent in the fleet runs on a model with a 1M-token context window. The operator layer rotates a session at 650K tokens, before the client's own (native) compaction, which starts at about 950K. Earlier internal notes called 650K "the window". It is the rotation threshold. Sessions in our sample reached 747,255 and 749,026 tokens before rotation, so the window cannot be 650K.

Measurement. We parsed 30 production transcripts: 15 from Node-Mac-x86 and 15 from the cloud nodes, spread over seven agents. One cloud transcript without usage data was skipped and replaced. We treat each model call as one step and read the context size from the usage the client logs for it (input plus cache creation plus cache reads). The growth between two calls is attributed in three parts. The model's own output is subtracted first, at its exact billed token count. Images get a fixed empirical cost, because their encoded size is not logged. The rest is split across the text items that entered the context in between (tool results, client attachments as rendered to the model, user-side text) in proportion to their characters. A compaction boundary, a negative jump or a model switch starts a new segment; such segments are counted but their reset is not attributed. Summing a session this way reconciled with its final context size to within 5% in 14 of 15 Node-Mac-x86 sessions (largest gap 7.7%, one attachment counted a call late) and within 0.03% in all 15 cloud sessions. A second pass over the raw transcripts, with an independent parser, matched the total growth to the token (7,942,054 and 10,490,761 tokens) and the shares to within 0.002 percentage points.

Result. Table 4 gives the full split.

Table 4: Sources of context growth over the life of a session, in percent of all tokens added, for the 30 sampled sessions. Shares are rounded; system messages (below 0.001%) are not shown.

Source of context growth	Node-Mac-x86	Cloud nodes
The model's own output, including thinking	39.4%	39.0%
Built-in tool results	29.7%	25.0%
— of which shell command output	18.8%	17.0%
— of which file reads	5.0%	7.2%
— of which sub-agent results	5.0%	0.5%
— of which other built-in tools	0.9%	0.3%
Client attachments (re-injected instructions, hook context, reminders, and messages queued while the agent was busy)	19.4%	14.3%
User-side text	7.7%	19.1%
— of which inbound channel messages	—	17.5%
Images	2.9%	1.9%
Responses of MCP tool servers	0.16%	0.71%
Unattributed	0.7%	0.0%
Total	**100%**	**100%**

Three readings matter for anyone budgeting a long-lived agent. First, the largest single consumer is the model itself. Thinking is not logged as text we can count: the transcripts keep a placeholder or a short summary, not the full reasoning. Our estimate is the part of the billed output that the visible text does not explain, converted at a measured characters-per-token ratio. It puts thinking at about 29% of output on Node-Mac-x86 and 42% on the cloud nodes. Since output is about 39% of all growth on both, that is about 11.5% and 16.3% of all growth. A separate calibration on Node-Mac-x86 gave 24% of output, or about 9.5% of growth. Because the estimate rests on that ratio, we treat the thinking share as uncertain. What we could test is that thinking from earlier turns is not dropped from the window: across six turn boundaries the context did not shrink by the expected amount. A large output share is also not waste. Most of it is the work itself. Second, "attachments" are not user files. They are the client re-injecting instructions and memory files, adding hook output, and holding messages that arrived while the agent was busy (3.5% and 4.0%); on Node-Mac-x86 the re-injected instructions alone are 6.55% of growth. Third, MCP responses are below one percent on both sides. In this configuration, cutting tool-server output, a common first move, would buy almost nothing.

After the exact output count and the fixed image cost, the remaining growth of each step is split in proportion to the characters each source added. That split is an estimate. The agreement of two parsers shows the calculation is reproducible, not that the split is exact, and it is least certain for thinking, which the transcripts do not count separately. All shares are of tokens *added over the*

session's life, not of the final window. On the cloud side the lifetime growth was 10.49M tokens against 6.16M net, because 22 compactions happened in the sample.

4.2 Idle compaction: a client mechanism found by its timing

Observation. In October several cloud agents compacted well before the native threshold, while others never did. An external model-based review noticed that the agents which compacted early ran a newer client version (2.1.295) and the others an older one (2.1.284).

What the timing showed. We re-ran the gap analysis over the transcripts from 11 to 14 October. On 2.1.295 we found 14 early compactions across four agents. Each came after 54.5–56.7 minutes without activity, with 213,197–563,598 tokens in context. The first was on 11 October. Over 5–15 October the three agents on 2.1.284 had no compactions in 96 comparable idle periods.

What the binary showed. The client contains a server-controlled idle-compaction flag. Its code defaults are a minimum of 200,000 tokens and a firing point at 0.9 of an interval, clamped to 0.5–0.95. Nine-tenths of the one-hour prompt-cache lifetime is 54 minutes, which fits the observed timing. The same flag and its event are also present in the 2.1.284 binary. Version 2.1.295 added only a user setting to turn the mechanism off, an environment variable for the threshold, and a new event name. The accurate statement is that the mechanism *activated* on 2.1.295, not that it was introduced there. Why it never fired on 2.1.284 we do not know. A server-side rollout by client version is our hypothesis, and that the server enabled the flag on 11 October is an inference from timing. The live server values were never observed; the timing fits the code defaults.

The client does not mark these events: in its own compaction metadata every one of the 21 compactions in the sample, idle or not, carries the same trigger value, "auto". Idle compaction can be told apart only by the idle time before it.

The 14 of 14 holds in one direction only. Every early compaction had the idle signature. Not every idle period produced a compaction: on 2.1.295, in the same window, 19 idle periods of at least 54 minutes above 200K tokens passed without one. The mechanism is therefore probabilistic or gated by something we did not observe, and "sessions above 200K compact after 54 minutes" is not a rule.

Mitigation. The fleet turned the setting off on the four cloud agents running 2.1.295 and on Node-Mac-x86. The three agents on 2.1.284 do not have the setting. After the change, three idle periods above 200K tokens passed without compaction. That is n=3 and weak evidence.

Lesson. A client update changed session lifetime with no change on our side, and the change showed up first as an anomaly in timing. Logging compaction events with the idle time before them made the cause visible. Without that log the early compactions would have looked like ordinary context pressure.

4.3 Study C: what a handoff brief returns after rotation and after compaction

Question. Long sessions in the fleet end in one of two ways. Either the operator layer *rotates* the session (a `/clear` before the native limit, followed by a handoff brief), or the client *compacts* the session natively and replaces the history with its own summary. ClaimKeep, the fleet's handoff layer, extracts verified claims, open threads and identifiers from the transcript into a brief. Study C asks how many checkable facts an agent can recover from each kind of context.

Pre-registration. The protocol, units, arms, hypotheses and scorer were committed before the main run. Two addenda followed: A1 after the pilot and before any main answer, A2 during the main run and before any scoring. Scoring and results were committed separately.

Deviation. One change was not declared as an addendum. After the pilot and before any main-run answer, the build script was corrected so that arm B contains only the first injected brief block. The change was committed one minute after the pre-registration and before addendum A1, but the protocol text does not mention it. We report it here as a deviation from the pre-registration.

Units and arms. Units are real production events, not synthetic sessions.

- *C1, rotation:* 21 rotations between 20 September and 4 October 2026 whose new session actually read the full brief file; 180 cloze questions. Arm A: empty context. Arm B: the head of the brief that is injected automatically at session start (at most about 9,000 characters). Arm C: the full brief file.
- *C2, native compaction:* 29 native compactions between 1 and 27 September 2026; 210 questions. Arm A: empty. Arm D: the client's native summary. Arm E: a ClaimKeep brief fitted under a character limit equal to the length of D; it may come out shorter. Arm F: D and E together.

Which models produced the contexts. The two regimes come from different weeks and different models. The C1 rotations (20 September to 4 October) all ran on Claude models: by the majority of assistant turns, 13 old sessions on Claude Opus 5.5, 6 on Claude Opus 5, and one each on Fable 5.1 and Sonnet 5.5. The C2 compactions (1 to 27 September) come from a period when part of the fleet ran other vendors' models through a router. At the compaction, the session ran a Claude model in 10 of the 29 units, GLM-5.3-flash in 10 and DeepSeek-v4.1-flash in 9. In this study, therefore, "native summary" means the client's summary mechanism in a mixed-model environment, not a summary written by one model. The client sends its compaction request through the same endpoint, so we take each summary to be written by the session's model, but the transcripts do not record its author directly. The brief lost to the native summary in each group: by 16.6 points where the session ran Claude, 17.9 with GLM and 10.9 with DeepSeek (exploratory, about ten units each). The answerers were Claude Opus 5.5 throughout.

Which events were excluded. C1 started from all 54 rotations in the window. In 23 of them the new session never read the full brief file, so arm C could not be built, and they were excluded. Another 8 gave fewer than five questions, and 2 of the remaining 23 were drawn as pilot units. C2 started from the 51 compactions kept by the frozen filters of an earlier natural-experiment study, which had kept 51 of 117 compaction events. Of these 51, 20 gave fewer than five questions and 2 were pilot units. The four pilot units (14 calls) were excluded from the analysis.

The C1 result therefore describes rotations in which the brief was actually read. In 43% of the rotations in the window (23 of 54) it was not, and the new session worked from the injected head alone, the condition of arm B. After the study the fleet reworded the pointer to the full file and added a reminder that repeats, up to ten times, until the file is read. We have not measured the read rate since.

Questions and answer keys. Questions were generated mechanically by the study's build script, with no model in the loop, from the agent's own text before the boundary: the old session for C1, the text before the compaction for C2. A probe extractor picks candidate items in four families (file paths, hashes and identifiers, facts written as a number next to a word, and claims), at most three per family and twelve per unit. Each question is the line the item came from, with the item masked as a blank; the answer key is the masked span. The build script and its seed are part

of the pre-registration commit. The script writes all units, questions and keys to one file, which the answering step then reads, and all arms of a unit get the same questions. That file is private and was checksummed only in the study package after the run, so the order of build before answers rests on the pipeline and the commit history, not on a checksum of the file. The answer files' modification times agree with that order: every answer was written after the pre-registration commit, the 14 pilot answers before addendum A1, and the first main answer 28 seconds after it.

The extractor is the benchmark's own code, separate from the brief builder, and it reads the transcript, not any arm. Two families still lean in opposite directions. Claim questions come from lines that carry a calibration marker, the same lines the brief builder keeps as claims, so they favour the brief. Fact questions come from bare numbers in prose, which the brief's extractor skips, so they work against it. The mix was 60 fact, 56 hash, 33 claim and 31 path questions in C1, and 78, 62, 44 and 26 in C2. Six claim keys (1.5% of all keys) kept a trailing full stop from their line, which an answer without the stop cannot match.

The shared character limit. In C2, D and E shared a character limit, not a size. The limit was the native summary's length in characters, because the brief's selector takes a character budget. A shared character limit means neither equal length nor equal tokens. The selector stopped below the limit: the brief used a median of 13.9K characters against 16.8K for the summary (median ratio 0.91, below 0.9 in 14 of 29 units). The two texts also differ in script. Cyrillic letters are 12% of the letters in the median summary and 41% in the median brief, and Cyrillic text costs more tokens per character. We did not count tokens per arm. The size shortfall does not explain the result in H4 below: in the 15 units where the brief reached at least 0.9 of the summary's length, it lost by 17.9 points; in the 14 where it stayed shorter, by 12.5. A comparison at matched token budgets is left for future work. Throughout this section, "same character limit" means exactly this.

Blind answering. Each (unit, arm) pair was answered by its own subagent (Claude Opus 5.5). The subagent could read exactly one file, holding that arm's context and the questions, and write exactly one answer file. It was told to answer only from the context and to say "unknown" otherwise. There were 193 such subagents, 179 in the main run and 14 in the pilot. In 192 of 193 calls the subagent used exactly the two permitted tool calls; one read its own long file twice, and its transcript was audited.

Scoring. The primary scorer is mechanical. An answer that is empty or says "unknown" counts as unknown. For a fact question the key number must appear in the answer as a whole number. For the other families both strings are normalised (case, quotes, backticks and whitespace removed), and the answer is correct if it contains the key. LLM judges were planned as a secondary measure and were not run. An independent node re-implemented the scorer from the protocol, without opening the original analysis code, and reproduced every primary number.

A containment rule can fail in both directions. It can credit a wrong answer that contains the key inside a negation, a list of alternatives or several numbers, and it can miss a correct answer written in another form. We checked both. First, mechanically: none of the 521 credited answers in the main run contains a negation word, an "or" between alternatives, more than one distinct number for a fact question, or text much longer than its key. Second, with a blind check of 114 answers: all 54 answers scored wrong, and 60 credited answers drawn at random, 15 per family. The arm was hidden and the items were shuffled. Two agents labelled the sample independently, not humans. The first was the cloud node, which did not take part in answering but had earlier re-implemented the scorer and knew its rules. The second was a separate agent on Node-Mac-x86 with a fresh context, which saw only the items and the labelling rule and not the first labels. All 60 credited answers were correct. That makes false credit rare in this data, though a sample of

60 cannot rule out a rate of a few percent. Of the 54 answers scored wrong, 16 were correct in another form: a path given without its leading directories, a name in Cyrillic transliteration, a date in another format, or a key with a trailing full stop. Four were partly correct and 34 were wrong. Rescoring with these labels raises accuracy by at most 2.3 points and changes no conclusion (Table 6). The two labellers agreed on 113 of 114 items (Cohen's $\kappa = 0.99$ over the four labels) and on all 114 once the labels are reduced to correct or not; the one disagreement was an accepted answer that one labeller called correct as written and the other correct in another form. Both labellings therefore give the same rescored numbers. The mechanical numbers stay primary; the rescored ones are a sensitivity check.

Statistical unit. Questions from the same event are not independent, so no test treats them as such. Accuracy is first computed per unit (one rotation or one compaction), and every comparison is a paired difference between two arms of the same unit. Confidence intervals come from 10,000 bootstrap resamples of units [3]; p-values are one-sided Wilcoxon signed-rank tests on the unit differences [11], adjusted across the five hypotheses with Holm's procedure [5]. Units are also nested in agents: C1 draws 21 units from 6 agents, and 12 of them come from one agent; C2 draws 29 units from 7 agents. An interval that resamples whole agents instead of units leaves the main conclusions in place: H1 +61.4 to +74.9 points, H4 −23.9 to −3.3. With six or seven clusters, one of them dominant, these agent-level intervals are rough. All intervals in this section come from the pre-registered analysis code; the independent re-implementation, with a different random-number generator, gave intervals within 0.2 points.

Results.

Table 5: Study C, answers by arm, in percent of questions averaged over units. "Wrong" is a non-empty answer the mechanical scorer did not credit. The last column rescores with the blind check.

regime	arm	units	questions	correct	wrong	unknown	correct, blind check
C1 rotation	A empty	21	180	0.0	0.0	100.0	0.0
	B brief head	21	180	21.5	2.0	76.5	22.3
	C full brief	21	180	70.7	4.3	25.1	73.0
C2 compaction	A empty	29	210	0.0	0.0	100.0	0.0
	D native summary	29	210	63.1	7.3	29.6	64.9
	E ClaimKeep brief, same character limit as D	29	210	47.8	4.5	47.7	48.4
	F D + E	29	210	67.6	7.1	25.3	68.8

Table 6: Study C, the five pre-registered hypotheses: mean paired difference per unit, one-sided Wilcoxon tests. The last two columns are sensitivity analyses: without questions whose key appears in another question of the same unit (addendum A2), and with the blind-check labels.

hypothesis	effect, points [95% CI]	Holm-adjusted p	outcome	without leaky questions	after blind check
H1 C1: C > A	+70.7 [63.8, 77.4]	0.00015	supported	+68.2	+73.0
H2 C1: C > B	+49.2 [43.0, 55.9]	0.00012	supported	+49.8	+50.7
H3 C1: B > A	+21.5 [15.7, 27.2]	0.00029	supported	+18.4	+22.3
H4 C2: E > D	−15.3 [−23.0, −7.3]	1.00	**rejected, in the opposite direction**	−20.7	−16.5
H5 C2: F > D	+4.5 [−0.8, 9.7]	0.095	not significant	+2.7	+3.9

Wrong answers stayed at or below 4.3% of questions in every rotation arm and at or below 7.3% in every compaction arm. Nearly a third of them (16 of 54) were correct answers in another form (see Scoring).

Leakage check. During the main run, an empty-arm subagent reported that some answers could be read off *other* cloze lines of the same unit, and declined to use them. Addendum A2 added a sensitivity analysis before any main answer was scored: drop every question whose key appears in another question of the same unit. On the main units this removed 100 questions. The conclusions did not change (Table 6): H1, H2 and H3 stay significant, and H4 and H5 stay non-significant in the claimed direction.

Interpretation.

1. *Rotation works when the brief is delivered whole.* The full file returns 70.7% of the probed facts; an empty start returns none. The automatically injected head returns 21.5%, under a third of what the full file returns. In this regime most of the measured loss sat in delivery: the full file held the facts, and the head that arrives on its own did not.
2. *Under the same character limit, the native summary beats our brief.* We expected the claim-structured brief to beat the client's summary under the same character limit, in a mixed-model setting where the summary's author varied (see above). It lost by 15.3 points. The per-family breakdown (Appendix A, Table 7) shows where: the brief lost most on file paths (15.4% against 53.8% for the native summary and 46.2% for both together), and did better only on explicitly stated claims. The brief's selector accounts for this pattern. Arm E was not cut from the end; it was fitted to the budget by the selector, which ranks items by weight. Paths and identifiers are stored as supplement items, and their maximum weight (0.42 for a path, 0.48 for an identifier) is below the minimum weight of any active claim (0.49). With a limit equal to the native summary's length, the selector spends the space on claims first, and paths and hashes get only what is left. We did not test a selector with other weights.
3. *Adding the brief to the native summary gives a small gain that does not survive correction* (+4.5 points, Holm-adjusted p = 0.095).

We read this as a negative result for the design goal "replace the native summary", and a positive one for "rotate early and deliver the whole brief". This is consistent with the operating rule the fleet already used from September on all cloud nodes and on Node-Mac-x86: rotate at 650K tokens, well before native compaction near the 1M window, and read the full brief as the first action of the new session.

What Study C does not show. The rotation result (70.7% against 0%) shows that delivering a full brief beats delivering nothing. It does not show that the brief's structure is what helps: at rotation the client writes no summary, so there is no same-size native summary to compare against, and building one would take a new run. The only test of structure we have is the compaction regime, and there our brief lost (H4). The study measures recall of probed facts, not whether the agent then works better. The 650K rotation threshold is an operating choice; this study does not test whether it is optimal.

Limitations. One fleet, one model family as answerer, cloze questions built from the same transcripts as the contexts. In this data the scorer's errors run mostly one way: it missed correct answers written in another form and, in the blind sample, credited no wrong ones. That is an observation on a sample checked by two agents, not a guarantee, so we do not call the mechanical accuracies lower bounds. The observational companion analysis of delivery and reuse (Study R) holds in direction only: its numbers depend on a "presence" rule the protocol did not fix, and its main test is not significant with the frozen scorer ($p = 0.11$). We do not report Study R numbers as findings.

Reproducibility package. The study's companion package lets a reader recompute every number above from per-unit counts. For each unit and arm it gives a pseudonymous unit and agent id, the regime, the number of questions, the numbers of correct, wrong and unknown answers by family, the context size in characters, and the rescored count from the blind check. It also holds the protocol with both addenda and the deviation note, the frozen scorer with its checksum, scorer tests on synthetic answers, and one script that rebuilds both tables, the bootstrap and agent-level intervals, and the Wilcoxon and Holm tests. The script reproduces the published numbers. Package: `https://github.com/rushnur88/what-breaks-in-long-lived-agents`; a copy is archived with this preprint at `https://doi.org/10.5281/zenodo.23404010`. Running `python3 rebuild.py` regenerates Tables 5 and 6, the intervals and the tests, and `python3 -m unittest discover -s tests` checks the output against the published numbers. The package cannot show whether the questions and keys were extracted correctly from the transcripts, because the transcripts are private. That is a separate level of reproducibility, and this package does not reach it.

5 Code Owns Control, a Small Model Judges, the LLM Plans

5.1 The split

A long-lived agent makes many small closed decisions: is this memory relevant, is this field the applicant's phone number, does this code change look safe enough to send to review. Routing each of them through the main language model is slow, costs context, and produces free text that code then has to parse. The fleet moved several such decisions to a small calibrated decision model and kept the control flow in ordinary code. The language model plans and writes; the decision model answers a fixed question with a label and a confidence; code decides what to do with the answer.

The decision model is a hosted classifier-style model (`jev-1.13`).[1] In 1,019 uncached production calls the provider reported the same model snapshot every time. Measured over the same calls, latency had a median of 444 ms, with 349 ms at the 10th and 600 ms at the 90th percentile; 69% of calls fell between 300 and 500 ms. On the routed path the price was $0.042 per million input tokens in every call, with output not billed. The physical nodes call the provider directly and fall back to a second route after a failure. The cloud host uses only the second route.

5.2 Where it is used, and how well it works

Memory relevance (live). Before recalled memories reach an agent's prompt, the decision model scores their relevance, and items below a per-agent threshold (0.3–0.5) are dropped. The threshold was set from blind labels: of recalled items scored below 0.2 by the retriever, 0 of 23 were useful. The filter has run live on all seven cloud agents since 23 September, with a daily calibration job. On 14 October it removed 80–83% of candidates. Its false-removal rate was 0–17%, but that rate is judged by another model, not by a person. Median latency on this path was 304–342 ms.

Form planning (laboratory). For job applications, the decision model maps each form field to an answer from a fixed bank or marks it "ask the human". Code fills the form, and a person confirms before anything is submitted. On a 15-field form the plan filled 11 fields and flagged 4, at a cost of $0.0061. On another node a 10-field run cost $0.0044.

Browser navigation (laboratory). A laboratory agent drives the browser from the visible page's DOM, extracted by script, without screenshots. It is not in production.

Code-review pre-screen (advisory, weak). A pre-screen asks the decision model whether a code change is likely to pass adversarial review. It never blocks. We matched each of the 48 later review verdicts (47 runs) with the last pre-screen call before it: 42 verdicts were "passed with warnings" and 6 were "blocked". Agreement was poor. Of the 42 passes, the pre-screen called 8 a pass, 27 a retry and 7 a fail; of the 6 blocks, it called 1 a pass, 3 a retry and 2 a fail. We keep it as an advisory signal and report it here as a weak result.

5.3 Honest negative results

The results below are the ones we can date and reproduce.[2]

A mood classifier that lost to "always neutral". We tested the decision model as a detector of the user's mood, against 81 blind labels from messages the labeller had not seen before. Accuracy was 0.64, with an expected calibration error [4, 9] of 0.097 (six unequal confidence bins with edges 0, 0.5, 0.7, 0.8, 0.9, 0.95, 1). A dictionary method scored 0.59. The trivial baseline "always neutral" scored 0.83. With a rule that falls back to "neutral" below confidence 0.8, the model reaches 0.83 on 38% coverage: it matches the baseline but does not beat it. The detector stays in shadow mode, logging but not acting, and the production default remains the dictionary.

Two smaller negative results. Bayesian Knowledge Tracing of per-domain mastery collapsed toward the model's bounds and was replaced in the practice layer by a smoothed success rate. An

[1] Model `typesafe/jev-1.13`, snapshot `jev-1.13-20260917`, from TypeSafe; reached directly and through OpenRouter. Prices and latencies are as observed in our logs from September to October 2026.

[2] An earlier automated run of the fleet's own hypothesis lab, on 4 April 2026, rejected 20 of 58 hypotheses. It was a single run in which "verified" meant "the check script passed", with no external replication, so we do not use it as evidence.

automatic process that proposed prompt rules inflated the rule set until an audit cut it back to twenty per agent. Both are documented in Appendix B.

5.4 What carries over

A small decision model is worth its place where the question is closed, the answer can be checked, and code keeps the right to ignore it. It failed where a trivial baseline was strong (mood) and where its question did not match the decision it was meant to predict (review pre-screen). In both cases we only learned this because the model ran in shadow, or in advisory mode, against labels collected blind.

6 Observed Operating Record

Most of this paper is about failures. This section reports what the fleet's own records say about its routine output, with the limits of those records next to the numbers.

6.1 The build pipeline

For larger changes the fleet uses a fixed pipeline: a brief, one owner checkpoint, a development agent, an adversarial review agent, and fix rounds when review rejects the work. Across 64 such phases recorded in the pipeline's state files on the cloud host and on Node-Mac-x86, the median time from the owner's go-ahead to an accepted review was about an hour: 51 minutes on Node-Mac-x86, and 58 minutes on the cloud host, where the go-ahead is recorded for 21 of 39 phases. In 6 of the 64 phases at least one review round rejected the work, and at least 8 needed a fix after they were closed. The set covers only phases kept in those state files; at least three further work packages on the cloud host ran outside them, and in each of them at least one first review round rejected the work. Review accepted the final work in 62 of the 64 phases, so acceptance says little on its own. In one phase a separate swarm of small models found three real defects that the reviewer had missed. The number of owner decisions after the go-ahead was extracted by an agent and not cross-checked: a median of zero, but up to ten in one website phase. These records were written by the agents; they measure the pipeline, not the value of its output.

6.2 The cloud host's task board

The cloud agents also keep a task board. It holds 795 cards created between June and mid-October 2026. Of these, 490 were closed one at a time by the agent doing the work, 201 were closed in bulk clean-ups (143 of them within one minute on 21 August, when board hygiene was introduced), 14 were removed by the hygiene rule for age, and 49 are open. The remaining 41 are marked closed but carry no completion time, so we cannot tell how they were closed. The individually closed cards were closed a median of about four minutes after they were created, and 85% within an hour. The agents mostly create a card at the moment they do the work. The board is therefore a work log written by the agents, not a queue of requests from the owner, and none of its cards records an approval step by the owner. It shows throughput. It does not show value to the owner.

7 Limitations

One fleet, one operator, one model family. Everything here comes from a single fleet run by one person. Nine of the ten agents run the same client, and most of the work ran on one model family. In September part of the fleet ran other vendors' models through a router at times, and 19 of the 29 compaction events in Study C come from such sessions (§4). The failures we report are real, but we cannot say how common they are elsewhere.

No denominator for false zeros. The false-zero cases in §2 are the ones we noticed. We do not know how many we missed, so the paper gives a catalogue and quantified examples, not a rate. The one place we tried to measure production precision, the Verify-Before-Deny gate, lost its own log for an unknown period.

Measurements from different months. Most numbers are from October 2026; some are older (the gate replay is from July, the hidden-file count from late July). Each row in the provenance table carries its date, and a number from one month says nothing about another.

Self-reported success inside the fleet. Task records, phase verdicts and board closures are written by the agents themselves. The fleet's task board, for example, records 795 cards, most of them created and closed by the agent doing the work within minutes, and none of them carries an approval step by the owner. These records show throughput, not value to the owner. We did not measure the owner's time saved, the cost per task, or whether a finished task was used.

Verification is internal. The second-node rule makes our numbers reproducible inside the fleet: a different agent re-derives each one with a different instrument. It does not make them reproducible outside. The transcripts and memory stores behind most numbers contain private messages and cannot be released. The companion package releases scripts, protocols and synthetic fixtures; the original data stays private. For Study C it also releases per-unit counts, from which every published Study C number can be recomputed, but not the questions and answers behind them (§4).

The authors are also the subject. The drafts of this paper were written by agents of the fleet it describes, under the author's direction, and the second-node checks, including the blind check of Study C answers, were done by those agents as well. We disclose this because it is the same echo-chamber risk that §3 warns about. Our mitigation is the provenance table: every number points to a primary file that a human can open.

Pipeline records are self-reported. The operating record in §6 comes from state files the agents wrote, extracted by an agent; part of it was cross-checked by the second node and part was not, as marked there.

Cost. The agents run on flat-rate subscriptions, and the records keep no per-task token or dollar count, so we cannot report cost per task or per result. The only costs we can state are those of the small decision model in §5, where every call is billed and logged.

8 Conclusion

The first lesson is that measurement comes before prompting. Most of the failures in this paper were not reasoning errors. They were instruments that returned a plausible empty answer: a permission boundary, a locale, a glob, an encoding, a filter. A model that reads `0` cannot tell an honest zero from a broken path, and no instruction in the prompt changes that. What helped was changing

the measurement path: a positive control in the same call, visible errors, and a gate that asks for evidence before a denial is written down.

The second lesson is that a check is only as independent as its position and its instrument. A reviewer that runs in the same environment, reads through the same tools and models the code instead of calling it agrees with the defect. In each of our cases the first check passed. The defect was found from a second horizon, by a second instrument, or by a test that satisfied the four conditions of §3.4.

The third lesson is about the context window. We did not run a comparison with less context or fewer rules, so we cannot say that more of either made the agents worse. What we measured is where the window went: to the model's own output and to re-injected instructions. The learned rules grew until an audit cut them back to twenty per agent. What held up in our operation was a lifecycle: rotate early, deliver the full brief, and move closed decisions to small calibrated models that code can overrule. Each of these held up only as far as we measured it, on one fleet, mostly on one model family. The negative results in §4 and §5 are part of the evidence.

Next experiments. Three experiments would turn the main observations into tested claims. First, controlled fault injection with the Verify-Before-Deny gate switched on and off, to measure how many false denials it prevents and at what cost in blocked correct answers. Second, a comparison of the claim-structured brief with a plain summary at the same token budget after rotation, the comparison Study C could not make, to test whether the structure of the brief adds anything beyond its delivery. Third, measuring whether real tasks continue more successfully after a handoff, so that recovered facts are tied to completed work and not only to answers on a quiz.

Data and Code Availability

The Study C reproducibility package (per-unit counts with pseudonymous ids, the protocol with its addenda and the deviation note, the frozen scorer, scorer tests and the rebuild script) is released with this preprint at `https://github.com/rushnur88/what-breaks-in-long-lived-agents` and archived at `https://doi.org/10.5281/zenodo.23404010`. It needs only Python 3. `python3 rebuild.py` regenerates Tables 5 and 6, the bootstrap and agent-level intervals and the Wilcoxon and Holm tests, and `python3 -m unittest discover -s tests` checks the result against the published numbers. Scripts and synthetic fixtures for the other measurements will be released in the same companion repository. Transcripts, memory stores and messages contain private data and are not released; the provenance table that ties each number to its primary file is kept by the author.

References

[1] Alan Agresti and Brent A. Coull. Approximate is better than "exact" for interval estimation of binomial proportions. *The American Statistician*, 52(2):119–126, 1998. doi: 10.1080/00031305.1998.10480550.

[2] Albert T. Corbett and John R. Anderson. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4):253–278, 1994. doi: 10.1007/BF01099821.

[3] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7 (1):1–26, 1979. doi: 10.1214/aos/1176344552.

[4] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017. arXiv:1706.04599.

[5] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues?, 2023. arXiv:2310.06770.

[7] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents, 2024. arXiv:2402.17753.

[8] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023. arXiv:2310.08560.

[9] Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 29, 2015. doi: 10.1609/aaai.v29i1.9602.

[10] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. arXiv:2303.11366.

[11] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. doi: 10.2307/3001968.

[12] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.10813.

[13] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023. arXiv:2306.05685.

A Study C, compaction regime: accuracy by question family

Each cloze question in C2 belongs to one family by the kind of answer it asks for: a file path, a hash or identifier, a stated fact (usually a number), or an explicit claim. Accuracy in percent of questions, pooled over the 29 units, mechanical scoring (Table 7).

Table 7: Study C, compaction regime: accuracy by question family, in percent of questions pooled over the 29 units (mechanical scoring).

family	D native summary	E ClaimKeep brief, same character limit	F D + E
path	53.8	15.4	46.2
hash	62.9	33.9	54.8
fact	61.5	48.7	71.8
claim	61.4	68.2	79.5

Under the same character limit the brief loses on every family except explicit claims. Adding the brief to the native summary helps on facts and claims and hurts on paths and hashes.

The losses on paths and hashes follow from how the brief is fitted to a character limit (§4.3). Every item gets a weight: an active claim at least 0.49, an identifier at most 0.48, a path at most 0.42. The selector keeps items in weight order until the budget is spent, so under a tight budget paths and identifiers are the first to go.

B Knowledge tracing and prompt-rule inflation

B.1 Knowledge tracing that collapsed to its bounds

The agents keep a per-domain mastery estimate. With Bayesian Knowledge Tracing [2], estimates were pulled to the model's bounds: in a 5 September snapshot of 91 domains, 16 sat at exactly 0.99 and 7 at exactly 0.16, and only 53 distinct values remained. For the practice layer we replaced it with the smoothed success rate (s+2)/(n+4), a Beta(2,2) prior, the same add-two-successes-and-two-failures adjustment as the Agresti–Coull interval [1]. It is not the Laplace estimate, which is (s+1)/(n+2). Against the observed success rate, mean absolute error fell from 0.2603 to 0.0367 on the snapshot, and the decision changed in 35 of 91 domains; a recomputation on 15 October gave 0.2595 and 0.0294. This comparison is close to circular, because the smoothed estimate is a shrunken success rate by construction. The stronger evidence is qualitative: the traced estimate could sit at 0.16 for a domain with an 88% success rate, and it correlated negatively with experience ($r = -0.24$).

B.2 Prompt rules that inflated

An automatic process proposed prompt rules from observed corrections. By 7 October the seven cloud agents carried 376 rules between them (50–61 each), of which 282 were unique. An audit kept 20 as core invariants, merged 50, moved 199 to other layers and dropped 13. Today each agent carries 20 rules (one carries 21). The reduction is 2.5–3× per agent, 2.7× across the fleet, and 14× counted by unique rules; the rule block shrank from 30–38 KB to 11 KB per agent.